

Source Code Management Lab Report

Name	Thanmay M Gowda
Enrollment No	A86605225219
Subject	Source Code Management (SCM)
Tool	Git & GitHub
Date	April 2026

1. Introduction

1.1 What is Git?

Git is a free and open-source **Distributed Version Control System (DVCS)** created by Linus Torvalds in 2005. It tracks changes in source code during software development, allowing multiple developers to work together on a project efficiently. Git records every modification made to files in a special database. If a mistake is made, developers can compare earlier versions of the code to help fix the error while minimising disruption to all team members.

Key features of Git include: branching and merging, distributed development, speed and performance, data integrity, and staging area.

1.2 What is Version Control?

Version Control (also known as source control) is the practice of tracking and managing changes to software code. Version Control Systems (VCS) are software tools that help software teams manage changes to source code over time.

Importance of Version Control in Software Development:

#	Benefit	Explanation
1	History Tracking	Every change is recorded with who made it and when.
2	Collaboration	Multiple developers can work on the same project simultaneously.
3	Backup & Restore	Files can be restored to any previous state at any time.
4	Branching	Developers can create isolated branches for features/bug-fixes.
5	Accountability	Provides an audit trail of all changes made to the codebase.

1.3 What is GitHub?

GitHub is a web-based platform for hosting and sharing Git repositories. It provides a cloud-based interface for version control and collaboration, allowing developers to work together on projects from anywhere in the world. GitHub was launched in 2008 and is now owned by Microsoft.

GitHub's role in collaboration: It provides features such as Pull Requests, Issues, Code Reviews, Actions (CI/CD), Wikis, and Project Boards that enable teams to develop software in an organised and transparent manner.

1.4 Difference Between Git and GitHub

#	Git	GitHub
1	A version control tool installed locally on a machine.	A cloud-based platform for hosting Git repositories.
2	Works without internet; purely a command-line tool.	Requires internet to access; provides a web UI.
3	Manages and tracks history of code changes.	Provides hosting, sharing, and collaboration features.
4	No user accounts or access control built-in.	Offers user accounts, teams, and permission management.
5	Open-source software maintained by the community.	A commercial service (free tier + paid plans) owned by Microsoft.

This section lists all Git/Git Bash and vi Editor commands performed during the lab, along with a description of what each command does and a screenshot of the output.

```
git --version
```

```
git config --global user.name "Thanmay M Gowda"
```

```
git config --global user.email "thanmay.logon@gamil.com"
```

```
git config user.name
```

```
git config user.email
```

```
git config --list
```

```
git help
```

Screenshot – Git configuration commands:

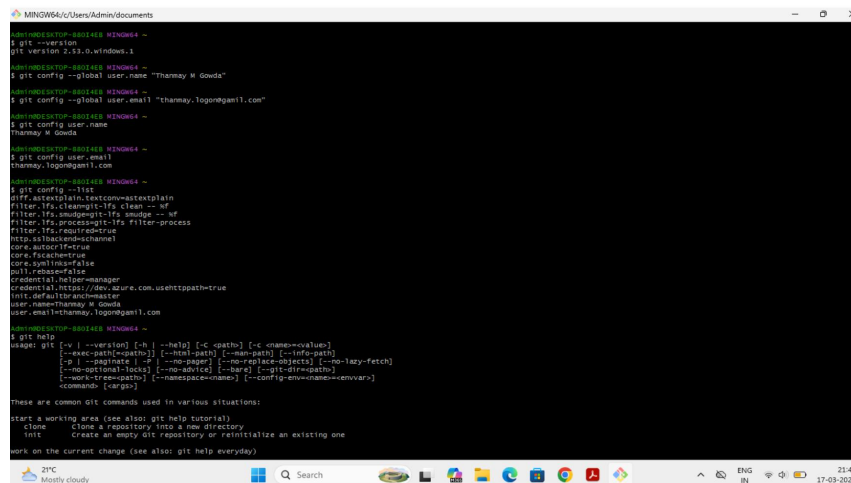
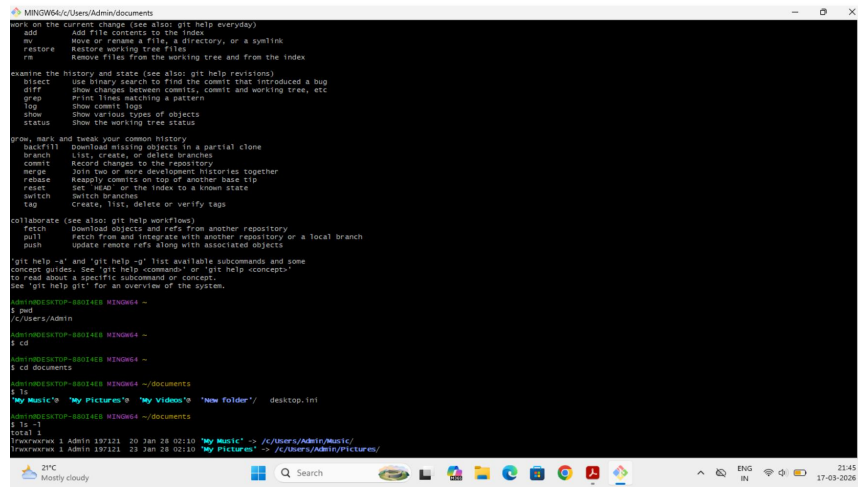


Fig 1: git --version, git config --global user.name/email, git config --list, git help

Screenshot – git help (continued) + navigation commands:



```
MINGW64/C:/Users/Admin/Documents
work on the current change (see also: git help everyday)
add      Add file contents to the index
mv       Move or rename a file, a directory, or a symlink
restore  Restore working tree files
rm       Remove files from the working tree and from the index

examine the history and state (see also: git help revisions)
bisect   Use binary search to find the commit that introduced a bug
diff     Show changes between commits, commit and working tree, etc
grep     Print lines matching a pattern
log      Show commit logs
show     Show various types of objects
status   Show the working tree status

grow, mark and tweak your common history
backfill Download missing objects in a partial clone
branch   list, create, or delete branches
commit   Record changes to the repository
merge    Join two or more development histories together
rebase   Reapply commits on top of another base tip
reset    Set HEAD or the index to a known state
switch   Switch branches
tag       create, list, delete or verify tags

collaborate (see also: git help workflow)
fetch    Download objects and refs from another repository
pull     Fetch from and integrate with another repository or a local branch
push     Update remote refs along with associated objects

'git help -a' and 'git help -g' list available subcommands and some
concept guides. See 'git help ' or 'git help '
to read about a specific subcommand or concept.
see 'git help git' for an overview of the system.

ADMINDESKTOP-BUIZHEB MINGW64 ~
$ pwd
C:/Users/Admin
ADMINDESKTOP-BUIZHEB MINGW64 ~
$ cd
ADMINDESKTOP-BUIZHEB MINGW64 ~
$ cd documents
ADMINDESKTOP-BUIZHEB MINGW64 ~\documents
$ ls
ADMINDESKTOP-BUIZHEB MINGW64 ~\documents
$ ls
My Music  My Pictures  My Videos  'new folder'  desktop.ini
ADMINDESKTOP-BUIZHEB MINGW64 ~\documents
$ ls -l
total 1
-rwxrwxrwx 1 Admin 197121 20 Jan 28 02:10 'My Music' -> /C:/Users/Admin/Music/
-rwxrwxrwx 1 Admin 197121 22 Jan 28 02:10 'My Pictures' -> /C:/Users/Admin/Pictures/
```

Fig 2: git help output (continued), pwd, cd, ls, ls -l

2.2 Git Bash – Navigation & File System Commands

```
pwd
```

Description: Prints the current working directory (Present Working Directory). Shows the full path of the directory you are currently in.

```
cd
```

Description: Changes the current directory to the home directory when used without arguments. Use 'cd ' to navigate into a specific directory.

```
cd documents
```

Description: Navigates into the 'documents' folder inside the current directory.

```
ls
```

Description: Lists all files and directories in the current directory (excluding hidden files).

```
ls -l
```

Description: Lists files and directories in long format, showing permissions, owner, size, and last-modified date.

```
ls -a
```

Description: Lists all files including hidden files (those starting with a dot '.').

```
ls -d */
```

Description: Lists only the directories (folders) in the current directory.

```
touch add.py
```

Description: Creates a new empty file named 'add.py'. If the file already exists, it updates the last-modified timestamp.

```
touch file.txt
```

Description: Creates a new empty file named 'file.txt'.

```
touch file.java file.html file.css file.cpp file.js file.xml file.md  
file.pdf file.json
```

Description: Creates multiple empty files at once with different extensions in a single command.

```
mkdir newfolder1 newfolder2 ... newfolder10
```

Description: Creates multiple new directories at once. 'mkdir' stands for Make Directory.

```
rm file.txt
```

Description: Removes (deletes) the file named 'file.txt' from the current directory permanently.

```
rmdir newfolder6
```

Description: Removes an empty directory named 'newfolder6'. Only works if the folder is empty.

```
rm -r hiiii
```

Description: Recursively removes a directory and all its contents (files and subdirectories). The '-r' flag stands for recursive.

Screenshot – File system commands (touch, mkdir, rm, rmdir):

The vi (visual editor) is a powerful text editor available in Unix/Linux systems and Git Bash. Below are the commands used during the lab.

Description: Opens (or creates if not existing) a file named 'thanmay.txt' in the vi editor.

i (Insert Mode)

Description: Switches vi from Command Mode to Insert Mode, allowing you to type and edit text. Press 'Esc' to return to Command Mode.

:w (Write/Save)

Description: Saves the file without exiting the editor. 'w' writes the buffer to disk.

```
:q (Quit)
```

Description: Exits the vi editor. If there are unsaved changes, vi will warn you.

```
:wq (Write and Quit)
```

Description: Saves the file and exits the vi editor in one command.

```
cat thanmay.txt
```

Description: Displays the contents of 'thanmay.txt' in the terminal. 'cat' stands for concatenate.

```
vi Thanmay.txt
```

Description: Opens or creates 'Thanmay.txt' (note: filenames are case-sensitive) in vi editor.

Screenshot – vi editor usage and cat command:

[illegible]

Fig 5: git config --list output, vi editor usage, cat thanmay.txt showing file content.

2.4 File Content Commands (echo, cat redirection)

```
echo "This is my first file" > file1.txt
```

Description: Writes the text 'This is my first file' into 'file1.txt', creating the file if it does not exist. The '>' operator overwrites the file content.

```
echo "Learning vi editor" >> file1.txt
```

Description: Appends the text 'Learning vi editor' to the end of 'file1.txt'. The '>>' operator appends without overwriting existing content.

```
cat file1.txt
```

Description: Displays the full contents of 'file1.txt' on the terminal screen.

```
cat > file2.txt
```

Description: Opens an interactive input mode – anything typed next is written into 'file2.txt'. Press Ctrl+D to save and exit.

```
cat file2.txt
```

Description: Displays the contents of 'file2.txt' to verify what was written.

```
cat file1.txt > file3.txt
```

Description: Copies the contents of 'file1.txt' into a new file 'file3.txt' using redirection.

```
cat file3.txt
```

Description: Displays the contents of 'file3.txt' to confirm the copy was successful.

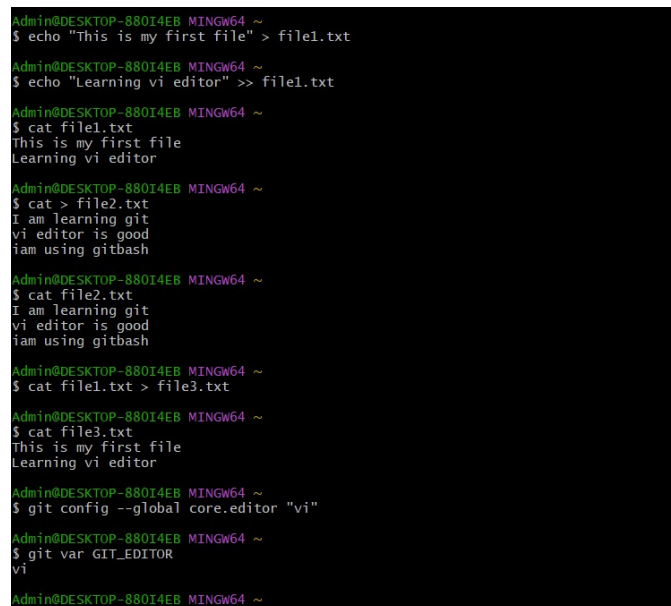
```
git config --global core.editor "vi"
```

Description: Sets vi as the default text editor for Git operations such as writing commit messages.

```
git var GIT_EDITOR
```

Description: Displays the currently configured Git editor. Confirms that 'vi' was set successfully.

Screenshot – echo, cat, file redirection, and git editor config:



```
Admin@DESKTOP-880I4EB MINGW64 ~
$ echo "This is my first file" > file1.txt

Admin@DESKTOP-880I4EB MINGW64 ~
$ echo "Learning vi editor" >> file1.txt

Admin@DESKTOP-880I4EB MINGW64 ~
$ cat file1.txt
This is my first file
Learning vi editor

Admin@DESKTOP-880I4EB MINGW64 ~
$ cat > file2.txt
I am learning git
vi editor is good
iam using gitbash

Admin@DESKTOP-880I4EB MINGW64 ~
$ cat file2.txt
I am learning git
vi editor is good
iam using gitbash

Admin@DESKTOP-880I4EB MINGW64 ~
$ cat file1.txt > file3.txt

Admin@DESKTOP-880I4EB MINGW64 ~
$ cat file3.txt
This is my first file
Learning vi editor

Admin@DESKTOP-880I4EB MINGW64 ~
$ git config --global core.editor "vi"

Admin@DESKTOP-880I4EB MINGW64 ~
$ git var GIT_EDITOR
vi

Admin@DESKTOP-880I4EB MINGW64 ~
```

Fig 6: echo commands, cat redirection, file1.txt / file2.txt / file3.txt, git core.editor

2.5 Command Summary Table

Command	Category	Purpose
<code>git --version</code>	Git Setup	Check Git version
<code>git config --global ...</code>	Git Setup	Set global username/email
<code>git config --list</code>	Git Setup	View all Git config settings
<code>git help</code>	Git Help	Show common Git commands
<code>pwd</code>	Navigation	Show current directory
<code>cd</code>	Navigation	Change directory
<code>ls / ls -l / ls -a</code>	Navigation	List directory contents
<code>ls -d */</code>	Navigation	List only directories
<code>touch</code>	File Ops	Create empty file(s)
<code>mkdir</code>	File Ops	Create new directory
<code>rm</code>	File Ops	Delete a file
<code>rmdir</code>	File Ops	Remove empty directory
<code>rm -r</code>	File Ops	Remove directory recursively
<code>vi <file></code>	vi Editor	Open file in vi editor
<code>i</code>	vi Editor	Enter insert mode
<code>:wq</code>	vi Editor	Save and quit vi
<code>cat <file></code>	File Ops	Display file contents
<code>echo "... " > file</code>	File Ops	Write text to a file
<code>echo "... " >> file</code>	File Ops	Append text to a file
<code>cat > file</code>	File Ops	Interactively write to a file
<code>cat f1 > f2</code>	File Ops	Copy file contents
<code>git config core.editor</code>	Git Setup	Set default Git editor
<code>git var GIT_EDITOR</code>	Git Info	Display current Git editor